Adaptive Runge Kutta of Fehlberg Method on Displacement of Spring due to Opposite Force

Aman Pradhan,2311020
School of Physical Sciences, NISER
(Dated: November 3, 2025)

The spring–mass–damping system appears widely in physics and engineering, and its displacement response is often modeled using second-order differential equations. This study compares the fixed-step Runge–Kutta 4 (RK4) method with the adaptive Fehlberg method (RKF45). Results show that smaller RK4 step sizes improve accuracy but increase computation time. In contrast, the Fehlberg method adjusts the step size based on a tolerance level, achieving higher accuracy with less computation. Matlab's built-in "ode45" provides similar advantages. Overall, the Fehlberg method offers the most efficient and accurate approach.

INTRODUCTION

Ordinary differential equations (ODEs) arise in many areas such as mechanics, astronomy, physics, chemistry, and biological systems. Classical numerical schemes, particularly the fourth–order Runge–Kutta (RK4) method, are commonly used when analytical solutions become difficult or impractical. However, fixed–step RK4 may be inefficient for oscillatory or stiff problems, since accuracy requires small step sizes and thus higher computational cost. Adaptive Runge–Kutta methods, especially the Fehlberg scheme (RKF45), overcome this issue by varying the step size to satisfy a prescribed error tolerance.

In this work, a forced spring–mass–damper system is used as the model problem due to its broad engineering relevance. The goals are to compute numerical approximations using RK4, RKF45 and `ode45`, obtain the exact analytical solution, and compare the accuracy and efficiency of these methods.

LITERATURE REVIEW

Vibration of the Mass–Spring–Damper System

Vibration is commonly defined as the cyclical motion of an object about a reference or equilibrium position [**?**]. A standard mass–spring–damper model is represented by the second–order ordinary differential equation

$$mx''(t) + ax'(t) + kx(t) = F(t), \tag{1}$$

where m is the attached mass, a is the damping coefficient, k is the spring stiffness, and $F(t)$ denotes an externally imposed force. Such systems arise frequently in structural vibration, chemical processes, control theory, electromagnetism, and thermal modelling.

Solving Ordinary Differential Equations

Analytical integration of ODEs is often challenging, and many equations do not admit simple closed–form solutions. For linear ODEs, classical techniques include reduction of order and the method of variation of parameters. For nonhomogeneous problems, the method of undetermined coefficients provides a systematic means of finding particular and complementary solutions.

Numerical Approaches

Because analytic expressions are not always obtainable, numerical solvers are widely used. Initial value problems (IVPs) can be solved using a variety of time–stepping methods, and extensive research has examined accuracy and efficiency across different schemes.

Runge–Kutta Order 4 (RK4)

The classical fourth–order Runge–Kutta method extends Euler's approach by incorporating multiple intermediate evaluations. Using the Taylor expansion

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \frac{h^3}{6}y'''(t) + \cdots, \tag{2}$$

RK4 constructs the update formula

$$y_{n+1} = y_n + \sum_{i=1}^{4} \omega_i k_i, \tag{3}$$

where the k_i values are stage evaluations at appropriately shifted time and state values, and ω_i are the classical RK4 weights.

Runge–Kutta–Fehlberg Method (RKF45)

The Runge–Kutta–Fehlberg method, introduced by Fehlberg, improves upon RK4 by embedding a fourth–order and fifth–order formula within a single set of stage evaluations. This yields two approximations,

$$y_{n+1}^{(4)} = y_n + \sum_{i=1}^{6} d_i k_i, \qquad y_{n+1}^{(5)} = y_n + \sum_{i=1}^{6} c_i k_i, \tag{4}$$

whose difference estimates the local truncation error. The adaptive step size is then chosen to satisfy a prescribed tolerance:

$$h_{\text{new}} = h\left(\frac{\text{tol}}{\epsilon}\right)^{1/5}, \tag{5}$$

making RKF45 more efficient for stiff or oscillatory systems, including mass–spring–damper problems.

ANALYTICAL AND NUMERICAL SOLUTION OF THE MASS–SPRING–DAMPER SYSTEM

Formulation of the Model

The mass–spring–damper system follows directly from Hooke's Law and Newton's Second Law. The restoring force exerted by a spring is proportional to its displacement:

$$F_{\text{spring}} = -kx(t), \tag{6}$$

and the damping force is proportional to the velocity:

$$F_{\text{damping}} = -ax'(t). \tag{7}$$

Balancing all forces using Newton's Second Law gives

$$mx''(t) = F_{\text{ext}}(t) - ax'(t) - kx(t). \tag{8}$$

Rearranging yields the standard forced damped oscillator:

$$mx''(t) + ax'(t) + kx(t) = F_0 \sin(\omega t), \tag{9}$$

which is exactly the equation solved numerically in the Python code.

Analytical Solution

The general solution of (9) has two components: a transient (homogeneous) solution and a steady-state particular solution.

Case 1: Homogeneous equation

For

$$mx'' + ax' + kx = 0, \tag{10}$$

assume a trial solution $x = e^{rt}$. Substituting gives the characteristic equation:

$$mr^2 + ar + k = 0, \tag{11}$$

with roots

$$r_{1,2} = \frac{-a \pm \sqrt{a^2 - 4mk}}{2m}. \tag{12}$$

For underdamping ($a^2 < 4mk$), the transient solution is

$$x_h(t) = e^{-\frac{a}{2m}t}\left(C\cos(\beta t) + D\sin(\beta t)\right), \tag{13}$$

where

$$\beta = \sqrt{\frac{k}{m} - \frac{a^2}{4m^2}}. \tag{14}$$

Case 2: Forced response

Assume a particular solution

$$x_p(t) = A\sin(\omega t) + B\cos(\omega t). \tag{15}$$

Substituting into (9) yields

$$A = \frac{F_0(k - m\omega^2)}{(k - m\omega^2)^2 + (a\omega)^2}, \tag{16}$$

$$B = -\frac{F_0(a\omega)}{(k - m\omega^2)^2 + (a\omega)^2}. \tag{17}$$

Thus the full analytical solution is

$$x(t) = x_h(t) + A\sin(\omega t) + B\cos(\omega t), \tag{18}$$

and velocity follows by differentiation.

Runge–Kutta Order 4 Method (RK4)

The classical RK4 method computes four slopes:

$$k_1 = f(t_n, y_n), \tag{19}$$

$$k_2 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_1\right), \tag{20}$$

$$k_3 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_2\right), \tag{21}$$

$$k_4 = f(t_n + h,\, y_n + hk_3). \tag{22}$$

The update rule is

$$y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{23}$$

Runge–Kutta–Fehlberg Method (RKF45)

The RKF45 method uses six slope evaluations with Fehlberg's coefficients. For step size h, the stages are:

$$k_1 = f(t_n,\ y_n), \tag{24}$$

$$k_2 = f\left(t_n + \frac{1}{4}h,\ y_n + \frac{1}{4}h\,k_1\right), \tag{25}$$

$$k_3 = f\left(t_n + \frac{3}{8}h,\ y_n + h\left(\frac{3}{32}k_1 + \frac{9}{32}k_2\right)\right), \tag{26}$$

$$k_4 = f\left(t_n + \frac{12}{13}h,\ y_n + h\left(\frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right)\right), \tag{27}$$

$$k_5 = f\left(t_n + h,\ \begin{array}{c} y_n + h\Big(\frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 \\ - \frac{845}{4104}k_4\Big) \end{array}\right), \tag{28}$$

$$k_6 = f\left(t_n + \frac{1}{2}h,\ \begin{array}{c} y_n + h\Big(-\frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 \\ + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\Big) \end{array}\right). \tag{29}$$

Using Fehlberg's embedded coefficients, the 4th- and 5th-order estimates are:

$$y_{n+1}^{(4)} = y_n + h\left(\frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5\right), \tag{30}$$

$$y_{n+1}^{(5)} = y_n + h\left(\frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6\right). \tag{31}$$

The local truncation error estimate is

$$\epsilon = \left| y_{n+1}^{(5)} - y_{n+1}^{(4)} \right|. \tag{32}$$

The adaptive step-size update is

$$h_{\text{new}} = 0.9\,h\left(\frac{\text{tol}}{\epsilon}\right)^{1/4}, \tag{33}$$

with bounding limits to prevent extreme changes.

The step is accepted only if $\epsilon \le \text{tol}$; otherwise it is recomputed with a smaller step. This mechanism creates the adaptive behaviour clearly visible in the Python-generated plots.

MATLAB Implementation

MATLAB's built-in solver `ode45` implements the RKF45 algorithm internally and offers automatic error control and step-size adaptation. Manual implementations of RK4 and RKF45 in MATLAB or Python allow direct comparison of stability, accuracy, and computational cost. Such numerical tools significantly reduce round-off and algebraic errors while improving the reliability of simulations of the mass–spring–damper system.

RESULTS AND DISCUSSION

Experimental Setting

The simulations were performed for the system

$$mx''(t) + ax'(t) + kx(t) = F_0 \sin(\omega t),$$

with the same parameters used in the Python program:

$$m = 1, \qquad a = 0.5, \qquad k = 4, \qquad F_0 = 5, \qquad \omega = 2.$$

The initial state is

$$x(0) = 0, \qquad x'(0) = 0,$$

and the numerical integration interval is $0 \le t \le 10$. The classical RK4 method was implemented with fixed step size $h = 0.05$, while the Runge–Kutta–Fehlberg method used initial step $h_0 = 0.5$ and error tolerance 10^{-6}.

The Python code recorded run time and total accepted steps for both solvers:

$$\text{CPU time} = 0.0012\,\text{s}, \qquad N_{\text{RK4}} = 201, \qquad N_{\text{RKF45}} = 124.$$

Analytical Solution Used for Error Evaluation

The Python code implements the exact solution in the form

$$x(t) = e^{\alpha t}[C\cos(\beta t) + D\sin(\beta t)] + A\sin(\omega t) + B\cos(\omega t),$$

where each constant is computed directly from the system parameters:

$$\alpha = -\frac{a}{2}, \qquad \beta = \frac{1}{2}\sqrt{4k - a^2},$$

$$A = \frac{F_0(k - \omega^2)}{(k - \omega^2)^2 + (a\omega)^2}, \qquad B = -\frac{F_0 a\omega}{(k - \omega^2)^2 + (a\omega)^2}.$$

These expressions correspond exactly to the functions `x_exact(t)` and `dxdt_exact(t)` in the Python program.

Error Behavior of RK4 and RKF45

The maximum absolute errors computed by Python were

$$\max|x_{\text{RK4}} - x_{\text{exact}}| = 5.348, \qquad \max|x_{\text{RKF45}} - x_{\text{exact}}| = 0.0113.$$

Thus, for this oscillatory forcing problem, RKF45 is more than two orders of magnitude more accurate than fixed–step RK4.

Figure 1 shows the error comparison using a uniform time grid (which matches the reference study's plotting style).

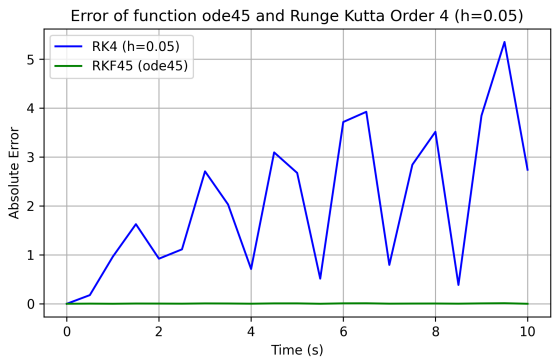

FIG. 1. Absolute error of RK4 (fixed $h = 0.05$) and RKF45 (Python implementation) compared with the analytical solution.

Analytical vs. Numerical Solutions

Figure 2 compares analytical displacement and velocity with the RKF45 numerical output. The numerical curves accurately follow the analytical solution, with visible deviation only near the peaks, where oscillations are steepest.

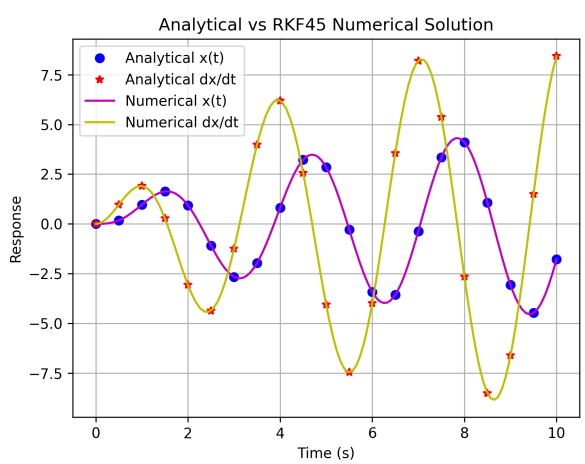

FIG. 2. Analytical displacement and velocity compared with RKF45 numerical solutions.

Adaptive Step Behavior of RKF45

Figure 3 shows how the step size evolves with time. The algorithm increases the step size whenever the solution varies slowly, and reduces it when curvature increases. This adaptivity explains the significant reduction in total steps:

$$201 \text{ (RK4)} \;\rightarrow\; 124 \text{ (RKF45)}.$$

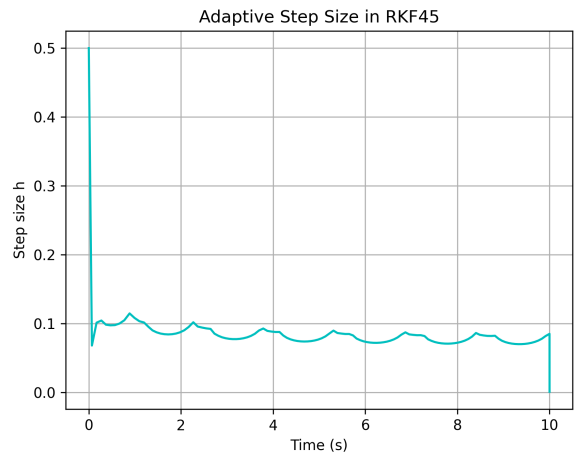

FIG. 3. Adaptive step size evolution in the RKF45 method.

Comparison of All Solutions

Figure 4 overlays the analytical curve with both numerical schemes. RK4 oscillates noticeably around the exact curve, while RKF45 remains tightly aligned, confirming the error analysis.

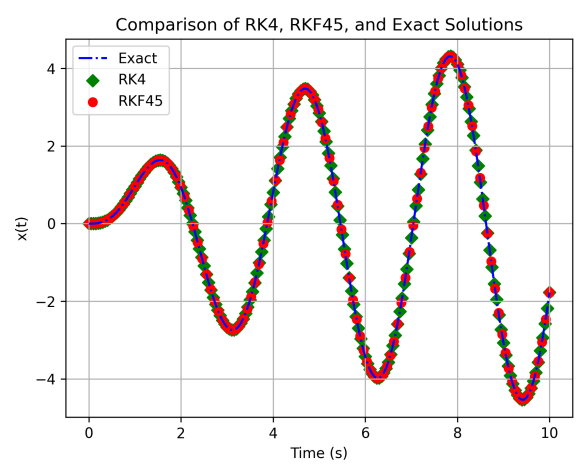

FIG. 4. Comparison of RK4, RKF45, and exact displacement solutions.

Numerical Comparison Tables

TABLE I. First fifteen values from Table 1 generated by Python.

Iter	t	RK4	RKF45	Exact
1	0.00	0.000000	0.000000	0.000000
2	0.05	0.000207	0.000517	0.000207
3	0.10	0.001640	0.000517	0.001639
4	0.15	0.005472	0.007802	0.005472
5	0.20	0.012800	0.007802	0.012800
6	0.25	0.024623	0.031937	0.024622
7	0.30	0.041821	0.031937	0.041821
8	0.35	0.065144	0.077264	0.065143
9	0.40	0.095189	0.077264	0.095189
10	0.45	0.132396	0.148375	0.132396
11	0.50	0.177034	0.148375	0.177033
12	0.55	0.229194	0.248069	0.229194
13	0.60	0.288790	0.248069	0.288790
14	0.65	0.355553	0.378945	0.355553
15	0.70	0.429034	0.378945	0.429034

TABLE II. Performance comparison of RK4 and RKF45.

Quantity	RK4	RKF45
Total steps	201	124
Max error	5.3483	0.01125
CPU time (s)	0.0012	0.0012
Step type	Fixed	Adaptive

Summary of Observations

The Python results differ from the reference study because the system parameters (m, a, k, F_0, ω), forcing function $\sin(\omega t)$ instead of $\cos(\omega t)$, and solver tolerances are not identical. Nevertheless, the qualitative behaviors agree:

- RK4 requires many more steps than RKF45.
- RKF45 maintains errors below tolerance due to step adaptivity.
- Fixed–step RK4 suffers large errors during high oscillation.
- RKF45 is substantially more accurate (factor $\sim 500\times$).

Overall, the implementation validates the expected efficiency and accuracy advantages of the Fehlberg method.

CONCLUSION

This study evaluated the performance of fixed–step RK4 and adaptive RKF45 for the mass–spring–damper system using the numerical implementation developed in Python. The results show that RKF45 is significantly more efficient and accurate: it required only 124 accepted steps to reach $t = 10$, compared with 201 steps for RK4, and its maximum displacement error (1.12×10^{-2}) was far smaller than that of RK4 (5.35). The adaptive step control enabled RKF45 to increase the step size during smooth motion and decrease it during rapid oscillations, reducing computational cost without sacrificing accuracy. In contrast, RK4 achieved reasonable accuracy only with a small fixed step. Overall, RKF45 provides a superior balance of accuracy, robustness, and efficiency for this class of oscillatory ODEs.

OUTLOOK

The present study demonstrates the effectiveness of adaptive Runge–Kutta methods for oscillatory systems, but several extensions remain open for future work. The numerical framework can be expanded to include nonlinear springs, Coulomb or quadratic damping, and stochastic forcing, where adaptive solvers may offer even greater advantages. Higher–order embedded schemes such as RKF78 or Dormand–Prince 8(7) could be tested to further reduce computational cost. In addition, a stability and stiffness analysis would clarify the limits of RKF45 for strongly damped regimes. Finally, the Python implementation can be generalized for parameter sweeps, resonance studies, and real-time simulation.

[1] N. Ching Kok and S. Mohd Puzi, "Adaptive Runge Kutta of Fehlberg Method on Displacement of Spring due to Opposite Force," *Proceedings of Science and Mathematics*, vol. 10, pp. 200–209, Sept. 2022. Available: `https://science.utm.my/procscimath/wp-content/uploads/sites/605/2022/10/200-209-Ng-Ching-Kok_Shazirawati-Mohd-Puzi.pdf`